

Research Report: Recent Feature Extractors and Retrieval Pipelines for VPR

Recommendations for the CMU-Africa Night-Time Campus VPR Project

Prepared for the current VPR tutorial codebase

April 24, 2026

Executive Summary

This report surveys recent **primary-source** VPR methods that are realistic candidates for this project. The goal is not to name the absolute best paper in isolation, but to identify the techniques that best match this codebase and project constraints:

- night-time and viewpoint variation,
- campus-scale maps,
- live localization and robot deployment,
- minimal engineering risk for near-term experiments.

Recommended top 3 for this project:

1. **SALAD (CVPR 2024)**: best first experiment overall. It is a strong single-stage global descriptor, has official Torch Hub support, and fits the current descriptor-based pipeline directly.
2. **BoQ / Bag-of-Queries (CVPR 2024)**: strongest raw benchmark contender among the surveyed drop-in descriptor methods, but with much larger descriptor sizes and therefore more expensive search.
3. **VPRTempo (ICRA 2024)**: best robotics-oriented efficiency option. It is not the most direct drop-in replacement for the current pipeline, but it is highly relevant if real-time CPU deployment becomes the priority.

Important nuance: if you want a purely *general-purpose, no-training* baseline, **AnyLoc-VLAD-DINOv2** is still one of the most useful methods to try. It is older than the 2024–2025 methods above, but it remains a very strong universal baseline and may be easier to test than VPRTempo.

How the Ranking Was Chosen

The ranking is based on the following project-specific criteria:

- **Performance relevance:** robustness to day-night, seasonal, or viewpoint changes.

- **Integration fit:** whether the method naturally plugs into the current pipeline, which expects a global descriptor from `compute_features()` and then runs similarity search.
- **Runtime fit:** whether the method can support a live or robot-facing demo.
- **Code availability:** preference for official code or official pretrained weights.
- **Recency:** emphasis on 2024–2025 methods, while keeping one older but still highly practical baseline where justified.

Why the Current Codebase Matters

Your current implementation is organized around **global descriptors**. In particular:

- `live_vpr/extractors.py` defines the supported descriptors and expects each extractor to return a 2D descriptor matrix.
- `live_vpr/offline.py` builds map descriptors from reference images.
- `live_vpr/online.py` compares a live query descriptor against saved map descriptors.
- `live_vpr/search.py` now supports multiple search backends, so larger descriptors are feasible, but they still increase memory and latency.

This means some new methods are **easy drop-ins** and some are **better treated as separate pipelines**. That distinction is central to the recommendations below.

Ranked Recommendations

1. SALAD (Optimal Transport Aggregation for Visual Place Recognition, CVPR 2024)

What it is. SALAD combines a fine-tuned DINOv2 encoder with an optimal-transport-inspired aggregation layer. The official repository describes two main ideas: using a fine-tuned DINOv2 encoder for richer features, and an aggregation method that extends VLAD with feature-to-cluster and cluster-to-feature assignments plus a dustbin for uninformative features.¹

Why it ranks first for this project.

- It is a **single-stage global descriptor** method, so it matches your current pipeline naturally.
- The official repo provides **Torch Hub** loading, which reduces integration friction.²
- The repo also exposes **multiple descriptor sizes** such as 512+32, 2048+64, and 8192+256, which is useful because you care about both accuracy and live-search cost.³
- It is one of the strongest recent **descriptor-style** methods, not just a reranker or a heavy two-stage system.

Why it is especially relevant to you.

¹<https://github.com/serizba/salad>

²<https://github.com/serizba/salad>

³<https://github.com/serizba/salad>

- Night-time robustness is likely to benefit from DINOv2-level features more than older CNN descriptors.
- The compact SALAD variants are attractive for **campus maps plus live search**.
- It can be tested in both `test_campus_dataset_place_level.py` and the live pipeline with minimal architectural changes.

Main caveats.

- DINOv2-based models are still heavier than your current CosPlace/EigenPlaces baselines.
- The official repo was tested with PyTorch 2.1, CUDA 12.1, and xFormers, so environment friction is possible.⁴

Bottom line. If you only try *one* new method first, try **SALAD**. It offers the best balance of modern performance, clean integration, and realistic deployment potential.

2. BoQ / Bag-of-Queries (CVPR 2024)

What it is. BoQ aggregates local features using a learned bag of queries and cross-attention, then projects the outputs into a global descriptor.⁵

Why it ranks second.

- The official repo provides pretrained **ResNet50+BoQ** and **DINOv2+BoQ** models via **Torch Hub**.⁶
- The released benchmark table is very strong, including **SVOX-night R@1 = 97.7** for the DINOv2-backed model.⁷
- Like SALAD, it produces a **global descriptor**, so it fits your current architecture directly.

Why it is not ranked first.

- The official DINOv2 model uses a **12288-dimensional** descriptor, and the ResNet50 model uses **16384 dimensions**.⁸
- Those descriptor sizes will make search, storage, and reranking more expensive in your live system.
- For a small benchmark this is fine, but for larger maps it creates more engineering pressure than SALAD.

Why it may still be worth trying.

- If your main goal shifts toward **best recall on benchmarks**, BoQ is arguably the strongest direct competitor to SALAD.
- Because you already added configurable ANN search backends, your codebase is better prepared for a high-dimensional descriptor than it was before.

⁴<https://github.com/serizba/salad>

⁵<https://github.com/amaralibey/Bag-of-Queries>

⁶<https://github.com/amaralibey/Bag-of-Queries>

⁷<https://github.com/amaralibey/Bag-of-Queries>

⁸<https://github.com/amaralibey/Bag-of-Queries>

Bottom line. BoQ is probably the best “push for maximum retrieval quality” experiment after SALAD, but it is heavier on the retrieval side.

3. VPRTempo (ICRA 2024)

What it is. VPRTempo is a temporally encoded **spiking neural network** for visual place recognition. The official project page describes it as an SNN designed for VPR with rapid training and fast query times for compute-limited robotic systems.⁹

Why it makes the top 3.

- It is the most clearly **robotics-first** method in this short list.
- The official page reports **over 50 Hz CPU inference** and support for datasets with **more than 27k unique places**.¹⁰
- The official package page exposes both a base model and an **quantized int8** variant for lighter deployment.¹¹

Why it is not ranked higher.

- **It is not a clean drop-in global descriptor extractor** in the same way SALAD or BoQ are.
- The method is place-centric and training-oriented: each output neuron is meant to respond to a place, so it fits better as a **map- or dataset-specific recognizer** than as a universal descriptor you can compare with cosine similarity.
- In other words, VPRTempo is highly relevant, but it should probably be treated as a **parallel experimental branch**, not just “another descriptor” in `live_vpr/extractors.py`.

Where it fits best in your project.

- A **TurboPi / live robot** branch where CPU speed matters more than leaderboard recall.
- A **map-specific** experimental setup where you train or adapt the model for a given traversal or environment.

Bottom line. If your next goal is **real-time robotics on modest compute**, VPRTempo is one of the most interesting methods to explore. If your next goal is simply to improve your existing descriptor pipeline, start with SALAD or BoQ first.

Other Techniques Worth Watching

AnyLoc-VLAD-DINOv2 (RA-L 2023 / ICRA 2024)

AnyLoc is still one of the strongest **general-purpose** VPR methods. The project page positions it as a universal VPR system that works across structured and unstructured environments without retraining or fine-tuning, and its repo explicitly presents AnyLoc-VLAD-DINOv2 as the SOTA configuration.¹²¹³

⁹<https://vprtempo.github.io/>

¹⁰<https://vprtempo.github.io/>

¹¹<https://pypi.org/project/vprtempo/>

¹²<https://anyloc.github.io/>

¹³<https://github.com/AnyLoc/AnyLoc>

Why it matters here. If you want an immediate experiment that uses **general foundation features** with **no VPR-specific training**, AnyLoc is arguably the safest research baseline to add.

Why it is not in the main top 3. It is older than the 2024 methods, and the engineering stack is somewhat heavier than a single ready-made hub model. But as a **universal baseline**, it remains very valuable.

EffoVPR (ICLR 2025)

EffoVPR is one of the most interesting recent papers because it argues that self-attention features from foundation models can serve as a powerful zero-shot reranker, and that internal ViT layers can produce **compact global features down to 128D**. The paper explicitly claims robustness under **occlusion, day-night transitions, and seasonal variation**.¹⁴

Why it matters here. It directly targets several of your pain points: compact features, strong generalization, and hard appearance changes.

Why it is not in the main top 3. Based on the primary sources reviewed here, it looks more like an excellent **research watchlist item** than the first method to integrate this week.

SuperVLAD (NeurIPS 2024)

SuperVLAD is presented as a **compact and robust** VPR descriptor built on DINOv2, with optional cross-image encoding.¹⁵

Why it matters here. Its positioning is attractive because compactness and robustness are exactly the tradeoffs you care about.

Why it is not in the main top 3. Compared with SALAD and BoQ, it looks a bit less battle-tested for immediate drop-in use, and the cross-image encoder adds complexity.

CricaVPR (CVPR 2024)

CricaVPR is a DINOv2-based representation-learning method for VPR with official code and pretrained weights.¹⁶

Why it matters here. It is a solid modern supervised VPR option.

Why it is not in the main top 3. It is more training-oriented than SALAD and less clearly deployment-oriented than VPRTempo.

Pair-VPR (RA-L 2025)

Pair-VPR is a **two-stage** ViT-based VPR system that first retrieves candidates with a global descriptor and then reranks them.¹⁷

Why it matters here. It is one of the more recent high-performance directions and aligns with your emerging interest in better re-ranking.

¹⁴<https://arxiv.org/abs/2405.18065>

¹⁵<https://github.com/lu-feng/SuperVLAD>

¹⁶<https://github.com/Lu-Feng/CricaVPR>

¹⁷<https://github.com/csiro-robotics/Pair-VPR>

Why it is not in the main top 3. The official repo is explicit that inference is GPU-heavy and training is expensive, making it much less practical for your current live robot timeline.¹⁸

Recommended Trial Order

If the goal is to improve the current codebase with minimal disruption:

1. **SALAD**
2. **BoQ**
3. **AnyLoc-VLAD-DINOv2**

If the goal is to improve the live robot story even at the cost of a separate code path:

1. **SALAD**
2. **VPRTempo**
3. **BoQ**

Concrete Recommendations for This Repo

Phase 1: easiest drop-in experiments

- Add **SALAD** and **BoQ** as new extractor wrappers under `feature_extraction/` and register them in `live_vpr/extractors.py`.
- Evaluate them first with:
 - `test_campus_dataset_place_level.py`
 - `test_campus_dataset.py`
 - your live map / live localization path
- Because BoQ descriptors are large, prefer the new ANN backends in `live_vpr/search.py` when testing live performance.

Phase 2: general-purpose foundation-model baseline

- Try **AnyLoc-VLAD-DINOv2** if you want to know how far a no-retraining foundation-feature pipeline can go on your campus.
- This is especially useful if you want a clean comparison against your manually relabeled place-level dataset.

Phase 3: dedicated robotics branch

- Explore **VPRTempo** as a separate experiment, especially for the TurboPi scenario.
- Do not force it into the current cosine-similarity descriptor pipeline unless you first verify that the package exposes a reusable embedding representation in the way your current code expects.

¹⁸<https://github.com/csiro-robotics/Pair-VP>

- A better framing is: **current pipeline** = descriptor retrieval, **VPRTempo branch** = direct place recognizer for resource-constrained deployment.

Final Recommendation

If your goal is **one strong next method to try in this repo**, choose **SALAD**. If you want the strongest second benchmark-oriented contender, add **BoQ**. If you want the most interesting robotics-specific branch for future deployment, investigate **VPRTempo**.

In short:

- **Best first practical upgrade: SALAD**
- **Best high-performance follow-up: BoQ**
- **Best edge/robotics research direction: VPRTempo**

Primary Sources Used

1. SALAD official repository: <https://github.com/serizba/salad>
2. BoQ official repository: <https://github.com/amaralibey/Bag-of-Queries>
3. VPRTempo official project page: <https://vprtempo.github.io/>
4. VPRTempo official package page: <https://pypi.org/project/vprtempo/>
5. AnyLoc official project page: <https://anyloc.github.io/>
6. AnyLoc official repository: <https://github.com/AnyLoc/AnyLoc>
7. EffoVPR arXiv page: <https://arxiv.org/abs/2405.18065>
8. CricaVPR official repository: <https://github.com/Lu-Feng/CricaVPR>
9. SuperVLAD official repository: <https://github.com/lu-feng/SuperVLAD>
10. Pair-VPR official repository: <https://github.com/csiro-robotics/Pair-VPR>